

UVC LED Buck Converter Driver for Water Disinfection

Project Summary

A closed-loop asynchronous buck converter driving an 8-LED Crystal IS Klaran UVC array for *E. coli* inactivation in water, designed and simulated in LTspice, with PI current control firmware on an Arduino controller.

Author	Aarav C Balaji
Advisor	Prof. Sanjiv Sambandan Chair, Dept. of Instrumentation & Applied Physics Chair, Flexible Electronics Lab Indian Institute of Science, Bangalore
Period	May 2026 – July 2026
Version	1.2

This document describes the design, simulation, firmware development, and hardware prototyping of a UVC LED driver for water disinfection research.

Contents

1	Introduction	2
2	System Specifications	2
2.1	Target LED – Crystal IS Klaran SA265-35030-SA1	2
2.2	LED Array Configuration	2
2.3	Converter Specifications	3
3	Circuit Design	3
3.1	Buck Converter Topology	3
3.2	Inductor Selection	3
3.3	Output Capacitor Selection	4
3.4	Gate Drive Circuit & Bootstrap Transient Analysis	4
3.5	Snubber Circuit	5
4	LTspice Simulation	5
4.1	Simulation Setup	6
4.2	Simulation Results	6
5	Firmware – PI Current Control	8
5.1	Timer1 PWM Configuration	8
5.2	ACS712-05A Current Sensing	8
5.3	PI Control Algorithm	8
6	Germicidal Efficacy	10
6.1	UV Dose Calculation	10
6.2	Dose-Response Data from Literature	10
7	PCB Design and Hardware Implementation	10
7.1	Complete KiCad Circuit Schematic	10
7.2	Two-Layer PCB Layout Architecture	11
7.3	Hardware Prototype Implementation	12
8	Conclusion	14

1. Introduction

Waterborne diseases caused by *Escherichia coli* and other pathogens remain a significant global health challenge. Ultraviolet-C (UVC) radiation in the 260 nm to 270 nm germicidal range disrupts bacterial DNA by forming pyrimidine dimers, preventing replication. A UV dose of 15 mJ/cm² to 20 mJ/cm² at 265 nm achieves a 4-log (99.99%) reduction of *E. coli* populations [1].

Solid-state UVC LEDs offer significant advantages over traditional low-pressure mercury lamps: instant-on capability, no hazardous mercury, compact form factor, and the ability to target specific germicidal wavelengths. However, they require precision constant-current drivers to maintain stable optical output and junction temperature within safe operating limits.

This report describes the complete design flow of a UVC LED buck converter driver: from specification and component selection, through LTspice simulation, Arduino firmware development, breadboard prototyping and PCB design, targeting the Crystal IS Klaran SA265-35030-SA1 UVC LED array.

2. System Specifications

2.1. Target LED – Crystal IS Klaran SA265-35030-SA1

The key electrical and optical parameters of the selected UVC LED are summarized below.

Table 1: Klaran SA265-35030-SA1 Key Parameters

Parameter	Symbol	Min	Typical	Max
Peak wavelength at 500 mA	λ_P	260 nm	–	270 nm
Forward voltage at 500 mA	V_F	5.0 V	–	9.0 V
Optical power at 350 mA	P_t	30 mW	–	–
Thermal resistance (junction-to-solder)	$R_{\theta JS}$	–	7 °C/W	–
Max junction temperature	T_J	–	–	115 °C
Continuous forward current	I_F	100 mA	–	700 mA

2.2. LED Array Configuration

Eight LEDs are arranged in a **2-series** $\times$ **4-parallel** (2S4P) configuration, chosen to balance string voltage against available supply headroom:

$$V_{\text{string}} = 2 \times V_F = 2 \times 7 \text{ V} = 14 \text{ V} \quad (\text{typical}) \quad (1)$$

$$I_{\text{total}} = 4 \times 500 \text{ mA} = 2 \text{ A} \quad (2)$$

$$P_{\text{optical}} \geq 8 \times 30 \text{ mW} = 240 \text{ mW} \quad (3)$$

2.3. Converter Specifications

The target design requirements for the buck converter operating in continuous conduction mode (CCM) are detailed below.

Table 2: Buck Converter Design Specifications

Parameter	Symbol	Value
Input voltage	V_{in}	18 V
Output voltage	V_o	14 V
Output current	I_o	2 A
Switching frequency	f_s	20 kHz
Duty cycle (M1)	D	77.8 %
Inductor ripple	ΔI_L	155.6 mA (7.8%)
Output voltage ripple	ΔV_o	≤ 0.35 V
Conduction mode	–	CCM (all cases)

3. Circuit Design

3.1. Buck Converter Topology

An asynchronous buck converter topology was selected, using an N-channel power MOSFET (IRLZ44N) as the main switch and a Schottky diode (1N5822) as the freewheeling element. The asynchronous approach was chosen for simplicity in the prototype phase; synchronous rectification may be considered in future iterations for improved efficiency.

The operating duty cycle is derived from the volt-second balance principle:

$$D = \frac{V_o}{V_{in}} = \frac{14}{18} = 0.778 \quad (77.8\%) \quad (4)$$

3.2. Inductor Selection

The critical inductance requirement for CCM operation is:

$$L_{crit} = \frac{(V_{in} - V_o) D}{2 I_o f_s} = \frac{4 \times 0.778}{2 \times 2 \times 20,000} = 38.9 \mu\text{H} \quad (5)$$

The inductor value for a target ripple current of $\Delta I_L = 10\%$ of $I_o = 2$ A:

$$L = \frac{(V_{in} - V_o) D}{f_s \Delta I_L} = \frac{4 \times 0.778}{20,000 \times 0.2} = 778 \mu\text{H} \approx 1 \text{ mH} \quad (6)$$

Verification of actual ripple with $L = 1$ mH:

$$\Delta I_L = \frac{(V_{in} - V_o) D}{L f_s} = \frac{4 \times 0.778}{0.001 \times 20,000} = 155.6 \text{ mA} \quad (7.8\%) \quad (7)$$

Since $L = 1 \text{ mH} \gg L_{\text{crit}} = 38.9 \mu\text{H}$, CCM is confirmed across all operating cases ($V_o = 10 \text{ V}$ to 14 V).

3.3. Output Capacitor Selection

Using the correct CCM ripple formula:

$$C = \frac{V_{\text{in}} D (1 - D)}{8 L f_s^2 \Delta V_o} = \frac{18 \times 0.778 \times 0.222}{8 \times 10^{-3} \times (20,000)^2 \times 0.35} = 0.62 \mu\text{F} \quad (8)$$

A value of $C_2 = 10 \mu\text{F}$ was selected, providing substantial margin above the minimum requirement and accounting for capacitor derating under DC bias.

3.4. Gate Drive Circuit & Bootstrap Transient Analysis

The IRLZ44N requires adequate V_{GS} enhancement for low $R_{DS(\text{on})}$. Because M1 operates as a high-side switch, its source voltage swings up to $V_{\text{in}} = 18 \text{ V}$ during conduction, necessitating a floating high-side gate driver.

In initial bench testing of a discrete bootstrap gate driver circuit, the node switching behavior was captured on an oscilloscope operating at $f_s = 19.98 \text{ kHz}$ (Figure 1). The captured waveform revealed severe transient ringing and voltage overshoot/undershoot at the switching transitions:

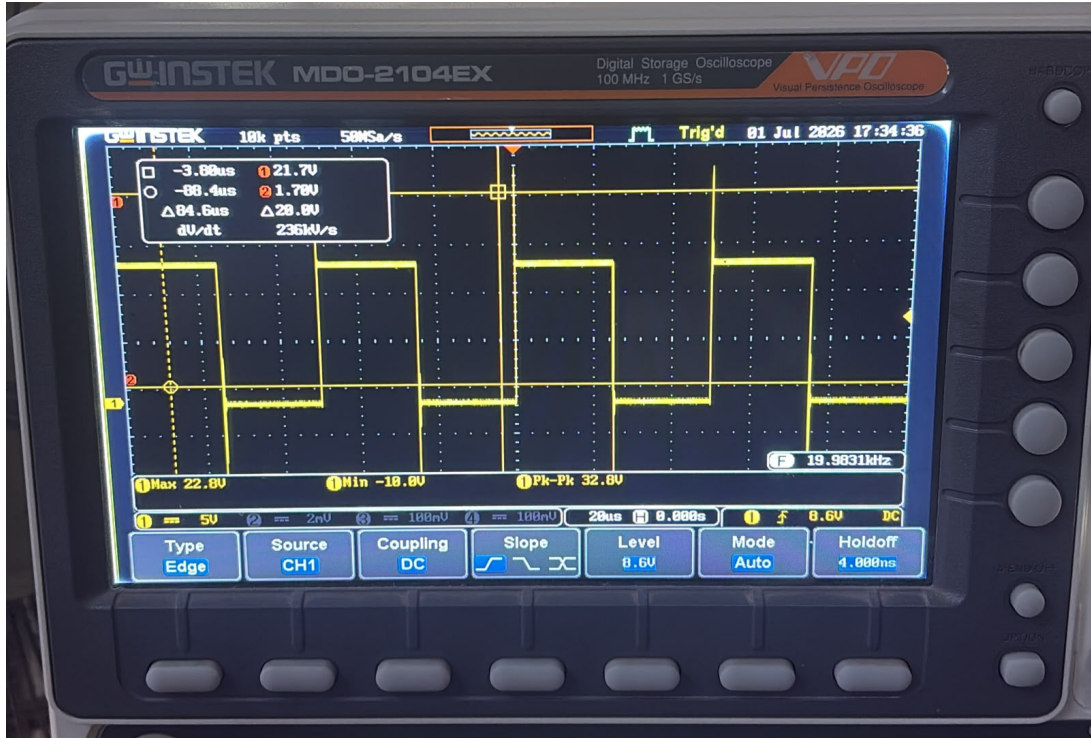


Figure 1: Oscilloscope trace of the gate switching node operating at 19.98 kHz, showing severe voltage overshoot spikes up to 22.8 V and negative undershoot down to -10.0 V ($V_{\text{pk-pk}} = 32.8 \text{ V}$) during discrete high-side switching.

As seen in Figure 1, high parasitic loop inductances and rapid dV/dt transitions ($\approx 236 \text{ kV/s}$) produced peak-to-peak voltage excursions of $\Delta V_{\text{pk-pk}} = 32.8 \text{ V}$, with maximum

voltage spikes reaching +22.8 V and negative ringing undershooting to −10.0 V. These severe spikes risk exceeding the maximum V_{GS} limits of the MOSFET and introduce high levels of electromagnetic interference (EMI).

To mitigate these transients and ensure clean, fast switching, a dedicated IR2110 high-side gate driver IC combined with an RCD snubber network was integrated:

- **Gate driver IC U2** (IR2110): provides a non-inverting, low-impedance push-pull high-side gate drive (HIN to HO) driven directly from a 5 V Arduino PWM signal.
- **Bootstrap diode D3** (1N4148): charges the bootstrap capacitor C1 to approximately $V_{in} - V_f = 18 - 0.7 \text{ V} = 17.3 \text{ V}$ during the MOSFET OFF period.
- **Bootstrap capacitor C1** (10 μF): selected such that $C_1 \gg Q_g / \Delta V_{gate}$ where $Q_g = 83 \text{ nC}$ for IRLZ44N.

$$C_1 \gg \frac{Q_g}{\Delta V} = \frac{83 \text{ nC}}{1 \text{ V}} = 83 \text{ nF} \quad \Rightarrow \quad C_1 = 10 \mu\text{F} \quad (9)$$

- **Gate resistor R1** (100 Ω): limits peak gate current and damps high-frequency ringing.

3.5. Snubber Circuit

An RCD snubber clamp was placed across the freewheeling diode D1 to suppress voltage spikes at the switching node. The network utilizes a fast-switching diode D2 (1N4148) in parallel with a bleed resistor R2 (12 k Ω) and in series with a capacitor C9 (30 nF).

During the main switch turn-on, D2 forward-biases, allowing C9 to quickly absorb high-frequency transient energy, which effectively clamps the ringing voltage. The capacitor then slowly discharges its stored energy through R2. The time constant of this discharge is:

$$\tau_{\text{snubber}} = R_2 \times C_9 = 12,000 \Omega \times 30 \text{ nF} = 360 \mu\text{s} \quad (10)$$

At a switching frequency of 20 kHz, the switching period is $T = 50 \mu\text{s}$. Because the snubber time constant is significantly larger than the switching period ($\tau = 7.2 \times T$), the capacitor does not fully discharge between cycles. Instead, it acts as a stable DC voltage clamp that tracks the transient peak.

The continuous static power dissipation in the bleed resistor is safely clamped:

$$P_{R2} \approx \frac{V_{in}^2}{R_2} = \frac{(18 \text{ V})^2}{12,000 \Omega} = 27 \text{ mW} \quad (11)$$

This confirms the chosen components operate well within the thermal limits of a standard quarter-watt (250 mW) resistor.

4. LTspice Simulation

4.1. Simulation Setup

Transient simulation was conducted in LTspice using the following control directive:

```
.tran 0 30m 0 100n
```

For full-power validation, the 8-LED UVC array was modeled by its equivalent full-load resistive load $R_3 = 7\ \Omega$ ($R_{eq} = V_o/I_o = 14\text{ V}/2\text{ A} = 7\ \Omega$).

To accurately capture high-side N-channel MOSFET driving dynamic behavior, the simulation incorporates a discrete driver stage with bootstrap acceleration. The PWM generator V2 driving the gate stage was configured as:

```
PULSE(0 5 0 10n 10n 11.1u 50u)
```

This directive sets a switching period of $T_s = 50\ \mu\text{s}$ ($f_s = 20\text{ kHz}$) and an active duty cycle of $D = 0.778$ (77.8%). In the LT spice image, IRFZ44N and 1N5818 have been used instead of IRLZ44N and 1N5822 as they do the same job and can be used interchangeably in the LT spice environment.

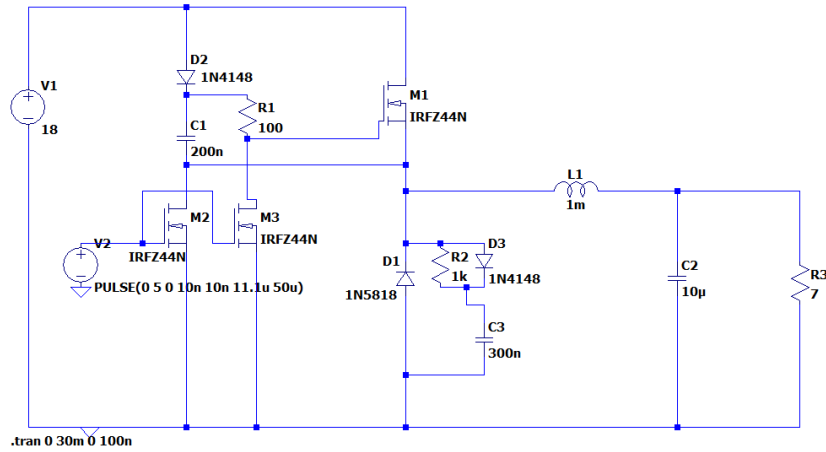


Figure 2: LTspice schematic of the asynchronous buck converter featuring the push-pull bootstrap gate drive circuit, snubber network, and a $7\ \Omega$ full-load resistor.

4.2. Simulation Results

The transient simulation results confirm steady-state performance against the initial design targets.

Table 3: LTspice Simulation Results vs Design Targets (Full Load)

Parameter	Target	Simulated	Status
Output voltage V_o	14.00 V	13.91 V (mean)	✓
Load current I_o	2.00 A	1.99 A (mean)	✓
Inductor ripple ΔI_L	$\leq 200\text{ mA}$	157 mA	✓
Output ripple ΔV_o	$\leq 0.35\text{ V}$	0.10 V	✓
Conduction mode	CCM	CCM confirmed	✓

As shown in Figure 3, the steady-state inductor current $I(L1)$ oscillates between 1.905 A and 2.062 A, giving a mean load current of 1.99 A and a peak-to-peak current ripple of:

$$\Delta I_L = 2.062 - 1.905 = 157 \text{ mA} \quad (12)$$

This closely aligns with the analytical target of 155.6 mA.

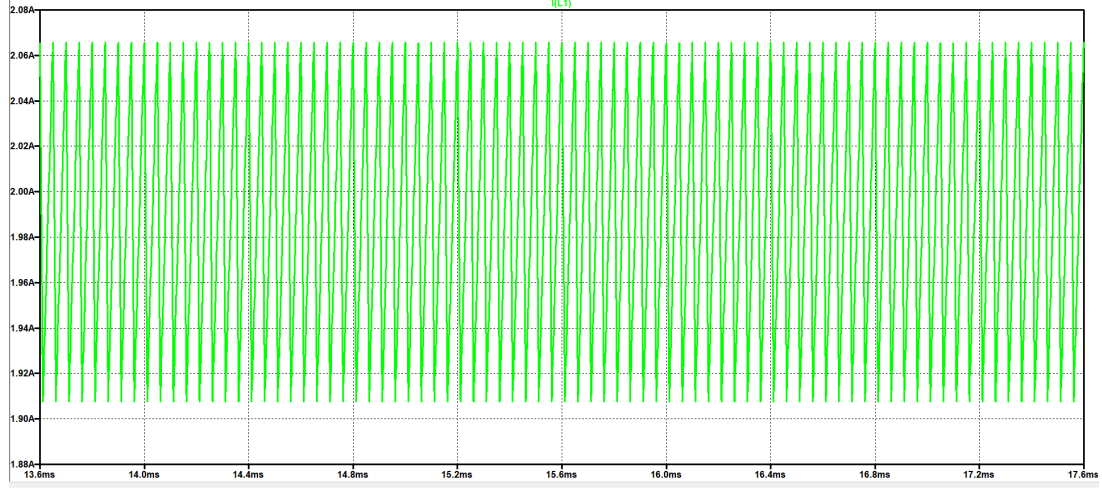


Figure 3: Simulated steady-state inductor current waveform (I_{L1}), demonstrating stable CCM operation with a mean current of approximately 1.99 A and a peak-to-peak ripple of 157 mA.

The steady-state output voltage waveform $V(n006)$ in Figure 4 settles at a mean of 13.91 V with peak-to-peak ripple bounds between 13.865 V and 13.965 V:

$$\Delta V_o = 13.965 - 13.865 = 100 \text{ mV} \quad (0.10 \text{ V}) \quad (13)$$

This minimal output ripple easily satisfies the design specification ($\leq 0.35 \text{ V}$).

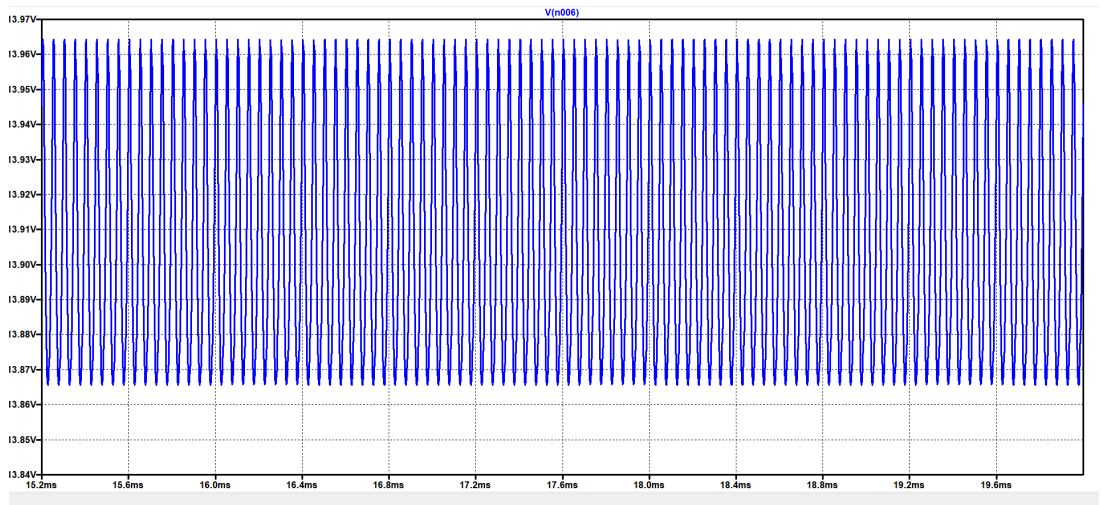


Figure 4: Simulated steady-state output voltage waveform (V_o), showing a stable average voltage of 13.91 V with a minimal peak-to-peak voltage ripple of 100 mV.

5. Firmware – PI Current Control

5.1. Timer1 PWM Configuration

Arduino Timer1 was configured in Fast PWM mode with ICR1 as the TOP value to generate the 20 kHz switching signal on pin 9:

$$f_{\text{PWM}} = \frac{F_{\text{CPU}}}{\text{ICR1} + 1} = \frac{16 \text{ MHz}}{799 + 1} = 20 \text{ kHz} \quad (14)$$

The starting OCR1A value for a 77.8 % non-inverting duty cycle:

$$\text{OCR1A} = D \times \text{ICR1} = 0.778 \times 799 \approx 621 \quad (15)$$

5.2. ACS712-05A Current Sensing

The ACS712-05A outputs a voltage proportional to current:

$$V_{\text{out}} = 2.5 + I \times 0.185 \quad [\text{V}] \quad (16)$$

$$I_{\text{mA}} = \frac{V_{\text{out}} - 2.5}{0.185} \times 1000 \quad (17)$$

At 500 mA per string (2 A total):

$$V_{\text{out}} = 2.5 + 2.0 \times 0.185 = 2.870 \text{ V} \quad (18)$$

5.3. PI Control Algorithm

```

1 //          Pin definitions

2 #define PWM_PIN      9
3 #define SENSOR_PIN  A0
4
5 //          PI tuning parameters (tune on bench)

6 float Kp = 0.1f;
7 float Ki = 0.01f;
8
9 //          PI state variables

10 float integral = 0.0f;
11 float pwm      = 621.0f; // start at 77.8% D (non-inverted: OCR1A
    =621)
12 float setpoint = 2000.0f; // target = 2000 mA total (4 x 500 mA)
13
14 //          Timing

15 unsigned long lastTime = 0;

```

```

16 int          loopCount = 0;
17
18 void setup() {
19     // Timer1: Fast PWM, ICR1 as TOP, no prescaler -> 20 kHz on pin 9
20     TCCR1A = _BV(COM1A1) | _BV(WGM11);
21     TCCR1B = _BV(WGM13) | _BV(WGM12) | _BV(CS10);
22     ICR1    = 799;    // f = 16 MHz / (799+1) = 20 kHz
23     OCR1A   = 621;    // D = 77.8% for IR2110
24
25     Serial.begin(115200);
26 }
27
28 void loop() {
29     //          Step 1: Read ACS712-05A on A0
30
31     int    raw          = analogRead(SENSOR_PIN);
32     float  voltage      = (raw / 1023.0f) * 5.0f;
33     float  currentmA    = ((voltage - 2.5f) / 0.185f) * 1000.0f;
34
35     //          Step 2: Error
36
37     float  error = setpoint - currentmA;
38
39     //          Step 3: Timing
40
41     unsigned long now = millis();
42     float dt = (now - lastTime) / 1000.0f;
43     lastTime = now;
44
45     //          Step 4: PI with integral windup clamp
46
47     integral += error * dt;
48     integral = constrain(integral, -1000.0f, 1000.0f);
49     float PI_output = (Kp * error) + (Ki * integral);
50
51     //          Step 5: Update PWM (non-inverting IR2110 logic)
52
53     pwm = pwm + PI_output;
54     pwm = constrain(pwm, 0.0f, 799.0f);
55     OCR1A = (uint16_t)pwm;
56
57     //          Step 6: Serial output every 10th loop
58
59     loopCount++;
60     if (loopCount >= 10) {
61         Serial.print(F("I_mA: ")); Serial.print(currentmA, 1);
62         Serial.print(F(" PWM: ")); Serial.print(pwm, 0);
63         Serial.print(F(" Err: ")); Serial.println(error, 1);
64         loopCount = 0;
65     }
66
67     delay(10); // 10 ms control loop
68 }

```

Listing 1: Arduino PI Current Control – UV LED Buck Converter Driver

6. Germicidal Efficacy

6.1. UV Dose Calculation

The delivered UV dose depends on irradiance and exposure time:

$$H = E \times t \quad [\text{mJ}/\text{cm}^2] \quad (19)$$

At 265 nm, the action spectrum for *E. coli* DNA damage peaks, making the Klaran SA265 LEDs near-optimally matched to the germicidal requirement.

6.2. Dose-Response Data from Literature

Table 4: *E. coli* Inactivation at 265 nm

UV Dose (mJ/cm ²)	Log Reduction	Kill Rate
5–6	1-log	90 %
10–11	3-log	99.9 %
15–20	4-log	99.99 %
25+	5-log	99.999 %

A minimum optical output of 240 mW from the 8-LED array targets 15 – 20 mJ/cm² at the water surface, sufficient for the 99.99 % kill rate design goal.

7. PCB Design and Hardware Implementation

To transition the design from simulation to physical prototyping, a custom double-sided Printed Circuit Board (PCB) was schematic-captured and routed using KiCad EDA.

7.1. Complete KiCad Circuit Schematic

The KiCad schematic in Figure 5 integrates power conversion, auxiliary regulation, precision gate driving, and signal sampling:

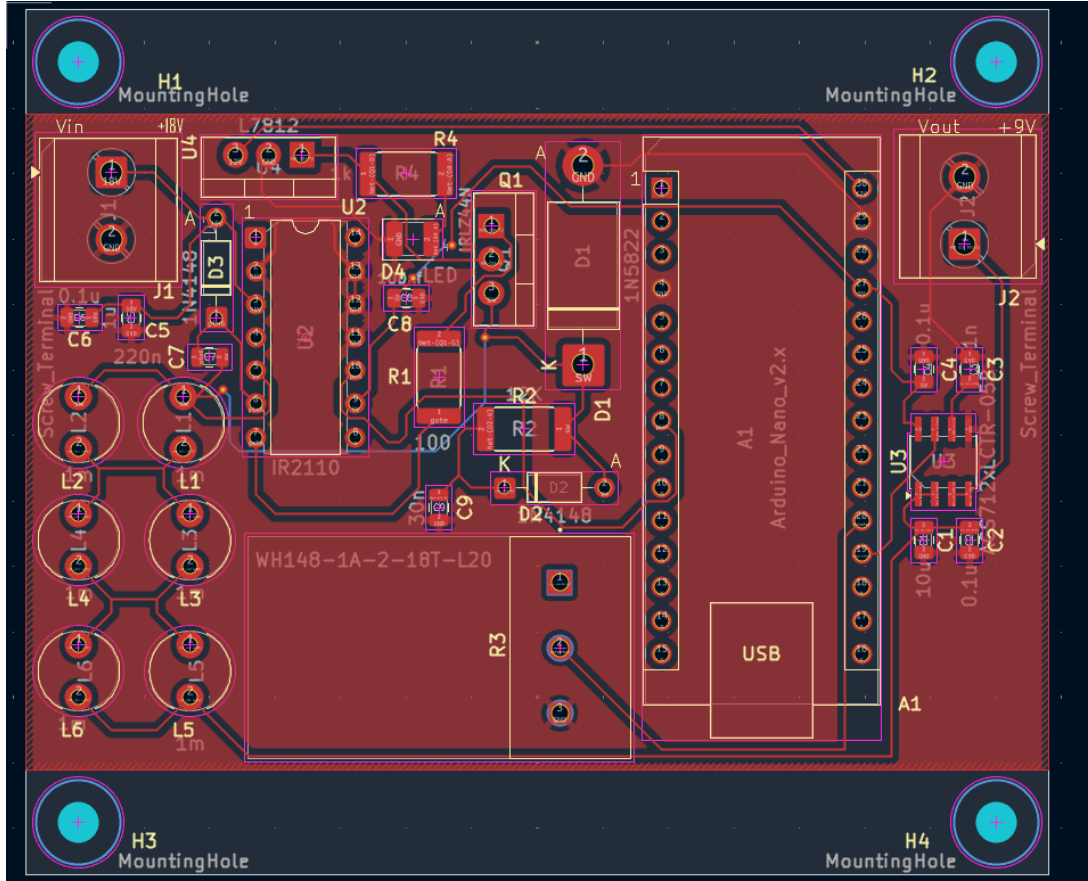


Figure 6: Two-layer PCB layout designed in KiCad, highlighting wide high-current power traces, thermal copper fills, isolated feedback signals, and standard M3 mounting holes.

7.3. Hardware Prototype Implementation

Prior to final PCB fabrication, the buck converter topology and closed-loop current feedback were verified on a breadboard setup (Figure 7). The hardware setup integrates the Arduino controller, switching power MOSFET, current feedback sensing circuitry, and passive components, allowing real-time calibration of the PI control parameters.

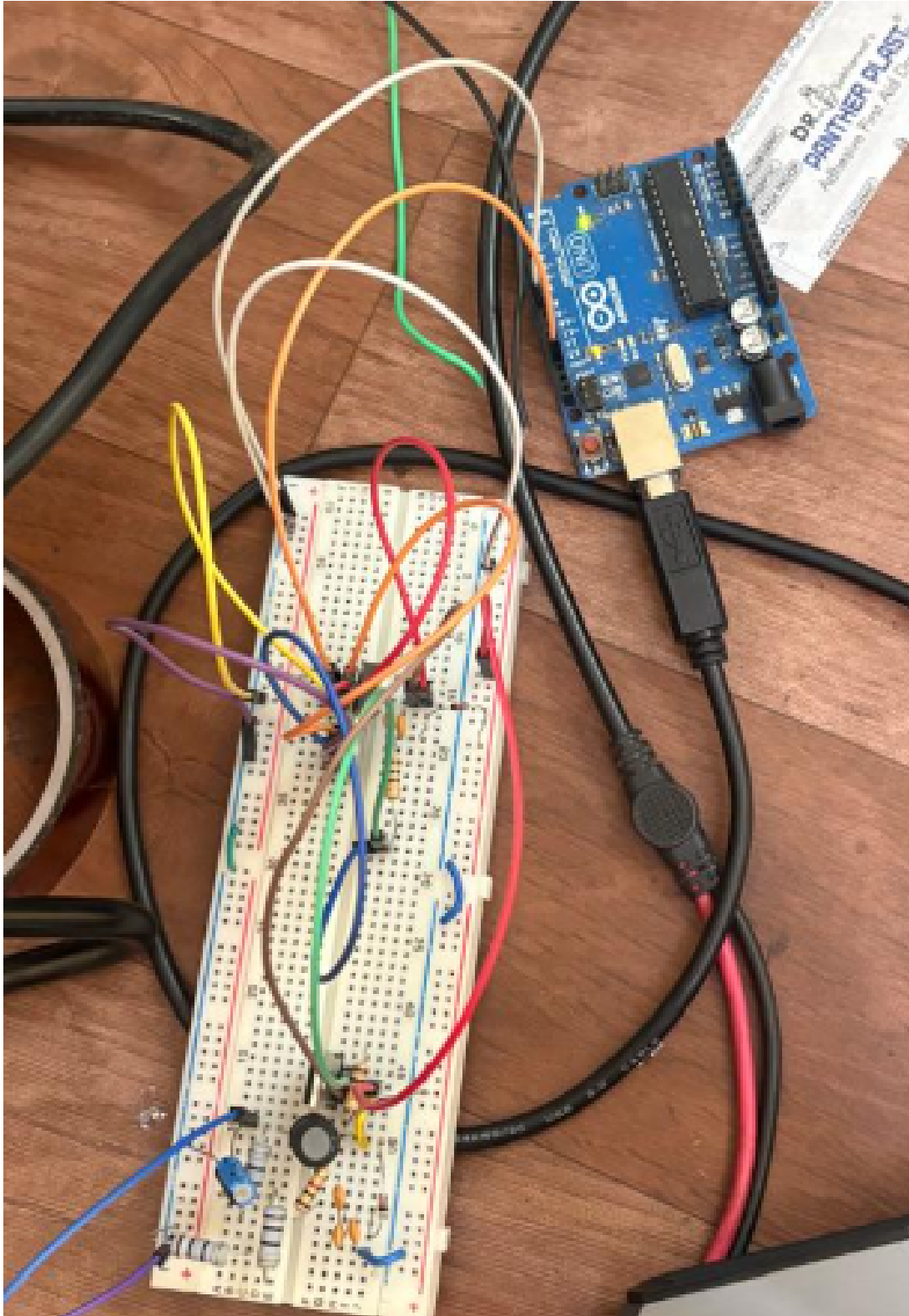


Figure 7: Breadboard hardware prototype setup of the UVC LED buck converter driver interfaced with an Arduino microcontroller during bench testing and parameter tuning.

8. Conclusion

A complete design flow was executed for a UVC LED buck converter driver targeting *E. coli* inactivation in water. The asynchronous converter ($18\text{ V} \rightarrow 14\text{ V}$, 2 A) was designed, simulated in LTspice, and validated to operate in CCM with a 7.8% inductor current ripple and $\Delta V_o = 0.10\text{ V}$ output ripple.

Bench measurements on a DSO revealed key high-side switching challenges ($V_{\text{pk-pk}} = 32.8\text{ V}$ ringing transients), justifying the implementation of an IR2110 driver IC and RCD snubber network. Closed-loop PI current control was implemented via Arduino firmware and ACS712 Hall-effect sensing. Finally, both breadboard prototyping and a custom 2-layer PCB layout were completed to ensure compact, reliable performance for water disinfection research.

References

- [1] World Health Organization, *Guidelines for Drinking-water Quality*, 4th ed. Geneva: WHO Press, 2017.
- [2] Crystal IS (Asahi Kasei), *Klaran SA Series UVC LEDs – Datasheet v2.0*, February 2024.
- [3] J. Neuhaeusler and M. Xie, “Different Methods to Drive LEDs Using TPS63xxx Buck-Boost Converters,” Texas Instruments Application Report SLVA419D, August 2021.
- [4] M. Würtele *et al.*, “Application of GaN-based ultraviolet-C light emitting diodes – UV LEDs – for water disinfection,” *Water Research*, vol. 45, pp. 1481–1489, 2011.